

# Python 编程与科学计算复习资料

数据处理、代码阅读与综合应用

## 使用说明

本资料围绕 Python 基础语法、文本提取、表格分析、数组计算、可视化和面向对象代码组织展开。学习时不要只背代码形式，要先弄清楚每一步操作处理的对象是什么、产生的结果是什么、为什么要这样写。

## 1 数据处理流程与表达

### 1.1 文本、文件或网页数据处理流程

**核心思路：**先明确目标，再获取数据、清洗数据、提取信息、保存结果并复核。

处理文本、文件或网页数据时，可以按下面的顺序组织思路：

1. **明确目标：**要提取哪些字段，要回答什么业务或学习问题，结果以什么形式保存。
2. **获取数据：**数据可能来自文本文件、CSV、网页响应、接口返回内容或调查表。
3. **基础清洗：**处理编码、空白字符、缺失内容、重复记录和明显异常值。
4. **提取信息：**可使用字符串方法、正则表达式、HTML 解析方法或表格读取方法。
5. **转换类型：**把数字字符串转为整数或浮点数，把日期字符串转为日期类型。
6. **保存结果：**保存为 CSV、文本、Excel 或图片等便于复核的形式。
7. **检查结果：**抽查样例、检查行数、比较统计值是否符合常识。

**可记忆表达：**

先明确数据处理目标和目标字段，再读取数据源；读取后进行编码、空白、缺失值和异常值处理；随后用字符串处理、正则表达式或解析工具提取所需信息；必要时转换数据类型；最后保存结果并检查样例和统计结果是否合理。

## 1.2 函数封装的作用

核心思路：函数把一段有明确目的的逻辑整理成可复用单元。

函数封装不是把简单事情复杂化，而是让程序更清楚、更容易复用和维护。常见作用包括：

- 减少重复代码，相同逻辑只写一次。
- 让程序结构更清晰，一个函数负责一个相对独立的任务。
- 用参数接收输入，用返回值交出结果。
- 便于单独测试某段逻辑。
- 便于定位错误，减少修改范围。
- 便于后续扩展，例如把一个计算步骤换成更复杂的规则。

```
def pass_rate(values, line=60):  
    count = len([x for x in values if x >= line])  
    return round(count / len(values), 2)  
  
rate = pass_rate([72, 58, 91, 66])
```

上面函数中，**values** 是输入数据，**line** 是判断标准，**return** 返回计算结果。以后标准发生变化时，只需要调整参数或函数内部逻辑。

## 1.3 数据分析前要明确的内容

核心思路：数据从哪里来、字段是什么意思、分析范围是什么、指标怎么算、结论如何复核。

| 准备项   | 说明                        |
|-------|---------------------------|
| 数据来源  | 文件、数据库、网页、问卷表单或日志等来源是否清楚。 |
| 字段含义  | 每一列代表什么，单位是什么，是否存在编码含义。   |
| 样本范围  | 数据覆盖哪些时间、对象、班级、地区或业务范围。   |
| 清洗规则  | 缺失值、异常值、重复记录如何处理。         |
| 分析指标  | 平均值、最大值、达标率、总量、占比等指标如何定义。 |
| 分组口径  | 按班级、时间、类别或其他维度分组时口径要一致。   |
| 可视化方式 | 趋势用折线图，关系用散点图，对比可用柱状图。    |
| 结果复核  | 解释结果时要说明样本范围、指标含义和限制。     |

## 2 正则表达式与文本提取

### 2.1 正则表达式是什么

正则表达式是一套用于描述字符串规律的写法，常用于查找、提取和验证文本。

例如，一段文本里可能混合了编号、姓名、分数、分隔符。普通字符串查找只能查固定内容，而正则表达式可以查“一类内容”。在 Python 中通常先导入 `re` 模块：

```
import re

text = "A12=86; A15=59; B20=91"
ids = re.findall(r"A\d{2}", text)
print(ids)
```

运行后，`ids` 会得到所有符合规律的编号。这里的重点不是某一个具体编号，而是理解 `A\d{2}` 这条规律。

### 2.2 `A\d{2}` 如何匹配

`A\d{2}` 表示：先出现字母 `A`，后面紧跟 2 个数字。

拆开看：

- `A`：普通字符，表示必须出现大写字母 `A`。
- `\d`：数字字符，相当于 0 到 9 中的任意一个数字。
- `{2}`：重复次数，表示前面的 `\d` 必须连续出现 2 次。

所以 `A\d{2}` 可以匹配：

- `A12`：`A` 后面有两个数字，可以匹配。
- `A08`：`A` 后面有两个数字，可以匹配。
- `A7`：`A` 后面只有一个数字，不能完整匹配。
- `A123`：前 3 个字符中的 `A12` 可被匹配，剩下的 3 不属于这次匹配。
- `B12`：开头不是 `A`，不能匹配。

```
import re

text = "A12=86; A08=74; A7=60; B12=91; A123=55"
print(re.findall(r"A\d{2}", text))
# 输出: ['A12', 'A08', 'A12']
```

注意最后一个结果来自 `A123` 的前三个字符。如果想要编号后面不能再接数字，可以进一步写边界规则，例如使用负向断言 `A\d{2}(?!\\d)`。

## 2.3 findall、分组和类型转换

**findall** 找到的是文本；如果要计算，通常要先转成数字。

`re.findall(pattern, text)` 会返回一个列表。没有括号分组时，返回整个匹配内容；有括号分组时，通常返回括号中捕获的内容。

```
import re

text = "S12=86; S15=59; S18=91"

labels = re.findall(r"S\d{2}", text)
numbers_text = re.findall(r"=(\d+)", text)
numbers = [int(x) for x in numbers_text]
passed = [x for x in numbers if x >= 60]
avg = round(sum(numbers) / len(numbers), 1)

print(labels)
print(numbers_text)
print(numbers)
print(len(passed), avg)
```

逐行理解：

- `labels` 提取形如 `S12` 的编号。
- `r"=(\d+)"` 中，`=` 表示等号本身，`\d+` 表示 1 个或多个数字。
- 括号 `(\d+)` 表示只把数字部分捕获出来，不要等号。
- `numbers_text` 中的元素还是字符串，例如 `"86"`。
- `int(x)` 把数字字符串转换成整数，之后才能求和、求平均值。

## 2.4 列表推导式与基础统计

列表推导式可以在一行中完成遍历、转换和筛选。

常见写法：

```
values = [86, 59, 91, 74]

high = [x for x in values if x >= 80]
as_text = [str(x) for x in values]
avg = round(sum(values) / len(values), 1)
top = max(values)
```

理解顺序：

- `for x in values` 表示逐个取出列表中的值。
- `if x >= 80` 表示只保留满足条件的值。
- `str(x)` 或 `int(x)` 表示对每个元素做类型转换。
- `sum`、`len`、`max` 常和列表配合计算总和、数量和最大值。

## 3 Pandas 表格筛选与定位

### 3.1 df[] 的常见用法

`df[]` 主要用于选列和按条件筛选行。

先构造一个接近学习记录场景的小表：

```
import pandas as pd

df = pd.DataFrame({
    "姓名": ["赵一", "钱二", "孙三", "周四"],
    "班级": ["C1", "C1", "C2", "C2"],
    "成绩": [82, 74, 91, 65],
    "学习时长": [6.5, 4.0, 8.0, 5.5]
})
```

`df[]` 有三种高频形式：

```
score_col = df["成绩"]
name_score = df[["姓名", "成绩"]]
high = df[df["成绩"] >= 80]
```

- `df["成绩"]`：取一列，返回 Series。
- `df[["姓名", "成绩"]]`：取多列，返回 DataFrame。
- `df[df["成绩"] >= 80]`：按条件筛选行，返回满足条件的 DataFrame。

条件筛选的关键是先生成布尔序列：

```
mask = df["成绩"] >= 80
print(mask)
print(df[mask])
```

`mask` 中每一行对应一个 True 或 False。把它放进 `df[mask]`，Pandas 就会保留 True 对应的行。

## 3.2 df.loc 的常见用法

**df.loc[行标签, 列标签]** 使用“标签”定位，不是单纯按第几个位置定位。

默认情况下，DataFrame 的行标签通常是 0、1、2、3。此时 `df.loc[1, "姓名"]` 表示取“行标签为 1、列标签为姓名”的值。

```
name1 = df.loc[1, "姓名"]
row1 = df.loc[1]
part = df.loc[1:2, ["姓名", "成绩"]]
```

需要注意：

- `df.loc[1, "姓名"]` 取单个单元格。
- `df.loc[1]` 取行标签为 1 的整行。
- `df.loc[1:2, ["姓名", "成绩"]]` 取行标签 1 到 2、指定两列。
- `loc` 的切片是按标签切片，右端标签通常会包含在结果中。

如果自定义了行标签，`loc` 的含义会更明显：

```
df2 = df.copy()
df2.index = ["stu01", "stu02", "stu03", "stu04"]

print(df2.loc["stu02", "成绩"])
print(df2.loc[["stu01", "stu03"], ["姓名", "班级", "成绩"]])
```

这里不能写 `df2.loc[1, "成绩"]` 来表示第二行，因为 `df2` 的行标签已经不是 0、1、2、3，而是 `stu01`、`stu02` 等。

## 3.3 df[] 与 df.loc 的区别

**df[]** 偏向“选列或筛行”，**df.loc** 偏向“按标签精确定位行列”。

| 写法                          | 适合做什么   | 示例                          |
|-----------------------------|---------|-----------------------------|
| <code>df[col]</code>        | 取一列     | 得到某一列数据                     |
| <code>df[cols]</code>       | 取多列     | 得到若干列组成的表                   |
| <code>df[mask]</code>       | 按条件筛选行  | 保留条件为 <code>True</code> 的记录 |
| <code>loc[row,col]</code>   | 按标签取单元格 | 得到指定行列交叉处的值                 |
| <code>loc[mask,cols]</code> | 筛选后再选列  | 得到条件范围内的指定列                 |

接近真实数据分析的写法：

```
good_names = df.loc[df["成绩"] >= 80, ["姓名", "班级", "成绩"]]
c1_mean = df.loc[df["班级"] == "C1", "成绩"].mean()
hours = df["学习时长"].tolist()
```

解释：

- 第一行先筛选成绩不低于 80 的记录，再只保留姓名、班级、成绩三列。
- 第二行先筛选 C1 班，再取成绩列并计算平均值。
- 第三行把学习时长这一列转成普通 Python 列表。

### 3.4 分组统计

**groupby** 是方法，要用圆括号调用；选列仍然用方括号。

```
mean_by_class = df.groupby("班级")["成绩"].mean()
max_by_class = df.groupby("班级")["成绩"].max()
count_by_class = df.groupby("班级")["姓名"].count()
```

常见错误是把方法调用写成方括号形式：

```
df.groupby["班级"]      # 错误
df.groupby("班级")      # 正确
```

原因是 **groupby** 是函数式方法，不是一个普通列名。

## 4 NumPy 数组与 Matplotlib 可视化

### 4.1 NumPy 数组统计与布尔索引

**NumPy** 数组可以直接做整体计算，也可以用条件筛选元素。

```
import numpy as np

arr = np.array([3, 5, 7, 9])
total = arr.sum()
avg = arr.mean()
selected = arr[arr >= 6]
idx = arr.argmax()
```

需要掌握：

- **arr.sum()** 求和，**arr.mean()** 求平均值。
- **arr >= 6** 得到布尔数组，例如 `[False, False, True, True]`。

- `arr[arr >= 6]` 根据条件筛选数组元素。
- `argmax()` 返回最大值所在的位置索引，索引从 0 开始。

## 4.2 常见图形选择

连续变化看折线图，两组数值关系看散点图，类别对比看柱状图。

```
import matplotlib.pyplot as plt
```

```
days = [1, 2, 3, 4]
values = [3, 5, 7, 9]
plt.plot(days, values)
plt.savefig("line_demo.png")
```

```
hours = [3.5, 5.0, 7.0, 8.5]
scores = [65, 72, 88, 93]
plt.scatter(hours, scores)
plt.xlabel("学习时长")
plt.ylabel("模块测评分")
plt.savefig("scatter_demo.png")
```

# 5 类、方法与常见错误

## 5.1 类的基本结构

类用于把数据和操作数据的方法放在一起。

```
class ResultTool:
    def __init__(self, values):
        self.values = values

    def average(self):
        return round(sum(self.values) / len(self.values), 1)

    def count_ge(self, line):
        return len([x for x in self.values if x >= line])

tool = ResultTool([76, 54, 83])
print(tool.average())
print(tool.count_ge(60))
```



逐点理解：

- `class ResultTool`: 定义一个类，冒号不能少。
- `__init__` 是构造方法，创建对象时自动执行。
- `self.values = values` 把传入的数据保存为对象自己的属性。
- 普通方法的第一个参数通常写 `self`。
- 调用方法时写 `对象.方法名()`。

## 5.2 常见错误检查

检查 Python 代码时，先看语法符号，再看方法名和逻辑条件。

常见错误包括：

- `class` 或 `def` 行末缺少冒号。
- 方法定义名和调用名拼写不一致。
- 筛选条件方向写反，例如要保留达标记录却写成小于标准。
- 把方法调用写成方括号，例如 `groupby["班级"]`。
- 应该保存对象属性时，忘记写 `self.`。

修正后的表格处理结构可以这样写：

```
import pandas as pd

df = pd.read_csv("learning_records.csv")
kept = df[df["成绩"] >= 60]
summary = df.groupby("班级")["成绩"].mean()
print(summary)
```

# 6 综合数据处理代码

## 6.1 读取、计算新列、分组统计、保存图像

综合数据分析代码通常按四段写：读文件、算新列、分组统计、画图保存。

下面示例使用一个课程过程性学习记录文件，字段包括编号、班级、平时练习、项目实践、模块测评、学习时长。代码展示如何计算综合表现、按班级统计，并绘制学习时长与模块测评分的关系图。

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("learning_records.csv")

df["综合表现"] = round(
    df["平时练习"] * 0.2 + df["项目实践"] * 0.3 + df["模块测评"] * 0.5,
    1
)

result = df.groupby("班级")["综合表现"].mean()
print(result)

plt.scatter(df["学习时长"], df["模块测评"])
plt.xlabel("学习时长")
plt.ylabel("模块测评")
plt.title("学习时长与模块测评关系")
plt.savefig("learning_scatter.png")
```

需要掌握的写法：

- 读取 CSV: `pd.read_csv("文件名.csv")`。
- 新增列: `df["新列名"] = 表达式`。
- 加权计算: 每个指标乘对应权重后相加。
- 分组平均: `df.groupby("分组列")["统计列"].mean()`。
- 散点图: `plt.scatter(x列, y列)`。
- 保存图像: `plt.savefig("图片名.png")`。

## 7 复习检查清单

1. 能否说清楚文本数据处理的一般流程。
2. 能否解释 `\d`、`{2}`、`\d+` 和括号分组的含义。
3. 能否判断 `findall` 返回的是完整匹配还是分组内容。
4. 能否区分 `df["列名"]`、`df[条件]` 和 `df.loc[行, 列]`。
5. 能否正确写出 `groupby("列名")["统计列"].mean()`。
6. 能否说明 NumPy 布尔索引和 `argmax()` 的结果含义。

7. 能否写出一个保存列表并提供统计方法的类。
8. 能否完成读取 CSV、计算新列、分组统计和保存图像的完整流程。

## 8 自测练习

### 练习一：正则提取与统计

```
import re

msg = "S12=63,S15=57,S18=90"
labels = re.findall(r"S\d{2}", msg)
nums = [int(x) for x in re.findall(r"=(\d+)", msg)]
good = [n for n in nums if n >= 60]
print(labels[1], len(nums), len(good), round(sum(nums) / len(nums), 1))
```

自查方向：先看每个正则表达式提取了什么，再看字符串数字是否已经转成整数。

### 练习二：Pandas 筛选与定位

```
import pandas as pd

df = pd.DataFrame({
    "姓名": ["甲", "乙", "丙"],
    "小组": ["G1", "G1", "G2"],
    "得分": [78, 92, 66]
})

print(df["得分"])
print(df[df["得分"] >= 80])
print(df.loc[1, "姓名"])
print(df.loc[df["小组"] == "G1", ["姓名", "得分"]])
```

自查方向：分别判断四个输出是列、筛选后的表、单个单元格，还是按条件定位后的局部表。

### 练习三：类与方法

尝试写一个类，保存一个数字列表，提供两个方法：一个返回保留一位小数的平均值，另一个返回不低于指定标准的元素个数。完成后用一个列表创建对象并调用两个方法。